UNIT-5

Dr.Y Srinivasa Rao

Department of CSE
MRU

December 29, 2025

Outline

Data Analysis

Numpy: Introduction to Numpy, creating arrays, using arrays and scalars, indexing arrays, array transposition, Array processing, Array Input and Output.
Pandas: Introduction to Pandas, Series in Pandas, Index objects, Reindex, Drop Entry, Select Entries, Data Alignment, Rank and Sort, Summary Statistics, Missing Data, Index Hierarchy.

What is NumPy?

- NumPy stands for **Numerical Python**.
- It is the fundamental package for **scientific computing** with Python.
- Provides a high-performance multidimensional array object, called the **ndarray**.
- Offers tools for working with these arrays, including linear algebra, Fourier transforms, and random number capabilities.

Why NumPy?

It is significantly faster than standard Python lists for numerical operations, thanks to vectorized operations.

Vectorized Operations

- What is a List?
- How to define a List.

Vectorized Operations

- What is a List?
- How to define a List.

Python Lists

list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9, 10]

Vectorized Operations

- What is a List?
- How to define a List.

Python Lists

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9, 10]
result = [x + y for x, y in zip(list1, list2)] # Slow for large data
```

Vectorized Operations

- What is a List?
- How to define a List.

Python Lists

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9, 10]
result = [x + y for x, y in zip(list1, list2)] # Slow for large data
```

NumPy arrays

```
import numpy as np
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([6, 7, 8, 9, 10])
```

Vectorized Operations

- What is a List?
- How to define a List.

Python Lists

```
list1 = [1, 2, 3, 4, 5]
list2 = [6, 7, 8, 9, 10]
result = [x + y for x, y in zip(list1, list2)] # Slow for large data
```

NumPy arrays

```
import numpy as np
arr1 = np.array([1, 2, 3, 4, 5])
arr2 = np.array([6, 7, 8, 9, 10])
result = arr1 + arr2 # Fast vectorized operation
```

Agenda

- What is a NumPy Array?
- Creating Arrays using `np.array()`
- 1D, 2D, and 3D Arrays
- Array Properties: `shape`, `dtype`, `ndim`

What is a NumPy Array?

- A NumPy array is a powerful data structure for numerical computing
- Stores elements of the same data type
- Faster and more memory efficient than Python lists
- Supports vectorized operations

Importing NumPy

```
import numpy as np
```

Creating Arrays with np.array()

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr)
```

- Converts Python list into NumPy array
- All elements have the same data type

1D Array (One-Dimensional)

```
import numpy as np


a = np.array([10, 20, 30, 40])
print(a)
```

- Single row of elements
- Similar to a Python list

2D Array (Two-Dimensional)

```
import numpy as np


b = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(b)
```

- Rows and columns
- Similar to a table or matrix

3D Array (Three-Dimensional)

```
import numpy as np


c = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
print(c)
```

- Collection of 2D arrays
- Used in images, videos, deep learning

ndim — Number of Dimensions

```
print(a.ndim)
print(b.ndim)
print(c.ndim)
```

- Shows how many dimensions an array has
- 1D $\rightarrow$ 1, 2D $\rightarrow$ 2, 3D $\rightarrow$ 3

shape — Size of the Array

```
print(a.shape)
print(b.shape)
print(c.shape)
```

- 1D: (number of elements,)
- 2D: (rows, columns)
- 3D: (blocks, rows, columns)

dtype — Data Type

```
print(a.dtype)

x = np.array([1.5, 2.3, 4.7])
print(x.dtype)
```

- Shows type of data stored in array
- Common types: int32, int64, float64

Summary

- NumPy arrays store numerical data efficiently
- `np.array()` creates arrays
- Arrays can be 1D, 2D, or 3D
- Important attributes:
  - `ndim` — dimensions
  - `shape` — size
  - `dtype` — data type

Quiz 1

What is the output of the following code?

```
import numpy as np
a = np.array([1, 2, 3])
print(a.ndim)
```

- A) 0
- B) 1
- C) 2
- D) Error

Answer 1

Correct Answer: B)

- The array is one-dimensional
- `ndim` returns number of dimensions

Quiz 2

What is the shape of the array?

b = np.array([[1, 2, 3], [4, 5, 6]])

print(b.shape)

- A) (3, 2)
- B) (6,)
- C) (2, 3)
- D) (2, 2)

Answer 2

Correct Answer: C)

- 2 rows and 3 columns
- Shape format: (rows, columns)

Quiz 3

Which attribute tells the data type of a NumPy array?

- A) `type`
- B) `ndim`
- C) `shape`
- D) `dtype`

Answer 3

Correct Answer: D)

`dtype` shows the type of elements stored in the array.

Day 3 Agenda

- What is Indexing?
- Indexing in 1D Arrays
- Indexing in 2D Arrays
- Slicing Arrays
- Modifying Array Elements

What is Indexing?

- Indexing is used to access array elements
- NumPy uses zero-based indexing
- First element has index 0

Indexing in 1D Arrays

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a[0])
print(a[2])
```

- a[0] $\rightarrow$ First element
- a[2] $\rightarrow$ Third element

Negative Indexing

```
print(a[-1])
print(a[-2])
```

- -1 accesses the last element
- Useful when array size is unknown

Indexing in 2D Arrays

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])

print(b[0, 1])
print(b[1, 2])
```

- Format: `array[row, column]`
- `b[0,1]` $\rightarrow$ 2

Slicing Arrays

- Slicing extracts a portion of the array
- Syntax: `start : stop : step`
- Stop index is excluded

Slicing in 1D Arrays

```
a = np.array([10, 20, 30, 40, 50])

print(a[1:4])
print(a[:3])
print(a[::2])
```

Slicing in 2D Arrays

```
b = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

print(b[0:2, 1:3])
```

- Selects rows 0–1 and columns 1–2

Modifying Array Elements

```
a[0] = 100
b[1, 1] = 999
```

- NumPy arrays are mutable
- Values can be changed directly

Day 3 Summary

- Indexing accesses single elements
- Slicing extracts subarrays
- Supports 1D and 2D operations
- Very fast and efficient

Practice Exercises

- Create a 1D array and access the last element
- Slice first three elements from an array
- Extract a sub-matrix from a 2D array

Array Transposition

- Transposition means converting rows into columns and columns into rows.
- It is mainly applied to 2D arrays (matrices).
- Represented as: A^T

Example of Array Transposition

```
A = np.array([[1, 2, 3], [4, 5, 6]])

transpose_A = A.T

print(transpose_A)
```

Key Points of Transposition

- Shape of array changes from (m, n) to (n, m)
- Does not change original data values
- Used in linear algebra and image processing

Quiz: Array Transposition

1. What does transposition of an array do?
2. Which NumPy attribute is used for transposition?
3. What will be the shape of a transposed (3, 2) array?

Array Processing

- Array processing involves performing operations on array elements.
- Operations can be:
  - Arithmetic
  - Logical
  - Statistical

Arithmetic Operations on NumPy Arrays

- Addition: `A + B`
- Subtraction: `A - B`
- Multiplication: `A * B`
- Division: `A / B`
- Modulus: `A % B`
- Power: `A ** 2`

Arithmetic Operations

```
import numpy as np

A = np.array([1, 2, 3])
B = np.array([4, 5, 6])

print(A + B)
print(A * B)
```

Logical Operations on NumPy Arrays

- Greater than: `A > B`
- Less than: `A < B`
- Equal to: `A == B`
- Not equal to: `A != B`
- Logical AND: `np.logical_and(A, B)`
- Logical OR: `np.logical_or(A, B)`

Statistical Operations on NumPy Arrays

- Mean (Average): `np.mean(A)`
- Sum: `np.sum(A)`
- Maximum value: `np.max(A)`
- Minimum value: `np.min(A)`
- Standard Deviation: `np.std(A)`
- Variance: `np.var(A)`

Statistical Operations

```
import numpy as np

A = np.array([10, 20, 30, 40])

print(np.mean(A))
print(np.max(A))
print(np.min(A))
```

Applications of Array Processing

- Signal processing
- Image processing
- Data analysis
- Machine learning

Array Input

- Arrays can be created by:
  - User input
  - Predefined values
  - Loading from files

Array Creation using User Input

```
import numpy as np
n = int(input("Enter number of elements: ")) arr = []
for i in range(n): arr.append(int(input()))
A = np.array(arr) print(A)
```

Using append() in Array Creation

- `append()` is used to add an element to the end of a list.
- It is commonly used when taking array input from the user.
- Values are first stored in a Python list.
- The list is later converted into a NumPy array.

```
arr = []
arr.append(10)
arr.append(20)
arr.append(30)
print(arr)
```

Array Creation using Predefined Values

```
import numpy as np
A = np.array([1, 2, 3, 4, 5])
print(A)
```

Array Creation by Loading from File

```
import numpy as np

A = np.load('data.npy')
print(A)
```

Array Output

- Arrays can be displayed using print()
- Can be saved to files using NumPy functions

Saving Array to File

```
import numpy as np

A = np.array([1, 2, 3, 4])
np.save('data.npy', A)
```

Quiz: Array Input and Output

1. How do you convert a Python list to a NumPy array?
2. Which function is used to save arrays to a file?
3. Which function is used to display array contents?

Summary

- Arrays are fundamental data structures in Python
- Transposition swaps rows and columns
- Array processing enables fast computations
- Input and output operations help store and retrieve data

Quiz: Array Processing

1. Which library is commonly used for array processing in Python?
2. What does np.mean() calculate?
3. Can arithmetic operations be applied element-wise on arrays?

What is Pandas?

- Pandas is a Python library for data manipulation and analysis .
- Built on top of NumPy , providing high-level data structures .
- Designed to work with structured data (tables, CSV, Excel, SQL).
- Two main data structures:
  - **Series**: 1D labeled array
  - **DataFrame**: 2D labeled tabular data

Why Use Pandas?

- Easy to load, clean, and analyze data.
- Supports missing data handling.
- Allows fast filtering, grouping, and aggregation.
- Integrates well with Matplotlib, Seaborn, and NumPy.

Installing and Importing Pandas

- Install using pip:

pip install pandas

- Import in Python:

import pandas as pd

Basic Pandas Data Structures

- **Series**: 1-dimensional labeled array

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)
```

- **DataFrame**: 2-dimensional labeled data

```
import pandas as pd
data = {'Name':['Alice','Bob'], 'Age':[25,30]}
df = pd.DataFrame(data)
print(df)
```

What is a Pandas Series?

- A **Series** is a one-dimensional labeled array capable of holding any data type.
- Labels for each element are called **index**.
- Built on top of NumPy arrays.
- Can hold integers, floats, strings, or Python objects.

Creating a Series from a List

```
import pandas as pd

data = [10, 20, 30, 40]
s = pd.Series(data)
print(s)
```

Creating a Series with Custom Index

```
import pandas as pd

data = [10, 20, 30, 40]
index = ['a', 'b', 'c', 'd']
s = pd.Series(data, index=index)
print(s)
```

Accessing Elements in a Series

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
print(s['y'])   # Access by index label
print(s[1])     # Access by position
```

Basic Operations on Series

- Arithmetic operations are element-wise

```
import pandas as pd

s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
print(s1 + s2)
```

- Statistical operations

```
print(s1.mean())
print(s1.max())
```

Pandas Index Object

- Every Pandas Series and DataFrame has an **Index** object.
- The index defines the labels for rows.
- Can be integers, strings, or dates.
- Provides fast lookup and alignment of data.

Accessing the Index of a Series

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s.index)    # Output: Index(['a', 'b', 'c'], dtype='
```

Reindexing a Series

- Reindex allows you to conform a Series to a new set of labels.
- Can add new indices or reorder existing ones.
- Missing values are filled with NaN by default.

Example: Reindexing

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
new_index = ['a', 'b', 'c', 'd']
s2 = s.reindex(new_index)
print(s2)
```

Reindex with Fill Value

```
s2 = s.reindex(new_index, fill_value=0)
print(s2)
```

- Missing index ‘d‘ is filled with 0 instead of NaN.

Dropping Entries in a Series

- Use the `drop()` function to remove elements by index.

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
s2 = s.drop('b')   # Drops index 'b'
print(s2)
```

Dropping Entries in a DataFrame

- `drop()` can remove rows or columns.
- Specify axis: `axis=0` for rows, `axis=1` for columns.

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = df.drop(1, axis=0)      # Drop row with index 1
df3 = df.drop('B', axis=1)    # Drop column 'B'
print(df2)
print(df3)
```

Key Points: Drop Method

- Returns a new object by default; original is unchanged.
- Can use `inplace=True` to modify the original object.
- Works for both Series and DataFrames.

Data Alignment in Pandas

- Pandas automatically aligns data **based on index labels** in arithmetic operations.
- Missing labels are filled with **NaN**.

```
import pandas as pd

s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])

print(s1 + s2)
# Output aligns on 'b' and 'c'; 'a' and 'd' get NaN
```

Rank and Sort in Pandas

Ranking assigns numerical ranks to data values based on their magnitude.

- Implemented using the `rank()` method
- Handles ties using different methods
- Supports ascending and descending order

Sorting arranges data by index or values.

- `sort_index()` – sorts by index labels
- `sort_values()` – sorts by column values

Rank and Sort – Example

```
import pandas as pd

s = pd.Series([100, 200, 150, 200])

# Ranking
s.rank()

# Sorting
s.sort_values(ascending=False)
```

Summary Statistics in Pandas

Summary statistics provide a quick numerical description of data.

- `count()` – number of non-null values
- `sum()` – sum of values
- `mean()` – average
- `min()` / `max()` – minimum and maximum
- `std()` – standard deviation
- `describe()` – complete statistical summary

Summary Statistics – Example

```
df.describe()
df.mean()
df.max()
```

Missing Data in Pandas

Missing data is commonly represented using `NaN`.

- `isnull()` / `notnull()` – detect missing values
- `dropna()` – remove missing values
- `fillna()` – replace missing values

Proper handling of missing data is essential for accurate analysis.

Missing Data – Example

```
df.isnull()
df.dropna()
df.fillna(0)
```

Index Hierarchy (MultiIndex) in Pandas

Index hierarchy allows multiple levels of indexing in rows or columns.

- Also known as **MultiIndex**
- Enables representation of higher-dimensional data
- Useful for grouped and hierarchical data

Index Hierarchy – Example

```
arrays = [['A', 'A', 'B'], [1, 2, 1]]
index = pd.MultiIndex.from_arrays(arrays)

s = pd.Series([10, 20, 30], index=index)
```

Advantages of Index Hierarchy

- Efficient representation of complex data
- Easy slicing and selection
- Powerful support for grouped operations

Conclusion

- Ranking and sorting help organize data
- Summary statistics provide quick insights
- Missing data handling ensures data quality
- Index hierarchy supports complex data structures

Python Lists: An Introduction

In Python, a list is a built-in data structure that can hold an ordered collection of items. Unlike arrays in some languages, Python lists are very flexible:

- **Can contain duplicate items**
- **Mutable:** items can be modified, replaced, or removed
- **Ordered:** maintains the order in which items is added
- **Index-based:** items are accessed using their position
- **Can store mixed data types** (integers, strings, booleans, even other lists)

Lists can be created in several ways, such as using square brackets, the `list()` constructor or by repeating elements. Let's look at each method one by one with example:

1. **Using Square Brackets**
   - We use square brackets `[]` to create a list directly.
2. Using the `list()` constructor